

IoT GSM-ENABLED SCROLLING LED DISPLAY

Arduino UNO | SIM900D GSM | MAX7219 ×8 | MD_MAX72XX | Proteus 8

Project Title	IoT GSM-Enabled Scrolling LED Display
Microcontroller	Arduino UNO (ATmega328P, 16MHz)
GSM Module	SIM900D (SoftwareSerial UART @ 9600 Baud)
LED Driver	MAX7219 × 8 (Daisy Chain SPI)
LED Matrix	8 × 8×8 = 512 LEDs (Scrolling Text)
Library	MD_MAX72XX by MajicDesigns
SMS Parsing	AT+CMGF=1 Text Mode, +CMT: Handler
Scroll Rate	150 ms per column (TSR Right-Shift)
Simulation	Proteus 8 Professional (COMPIM virtual port)
Build Targets	Arduino Nano + Arduino UNO (.hex provided)
Author	Shreyas Maharashtra, India

1. Abstract

This project presents an IoT-based GSM-Enabled Scrolling LED Display system that remotely updates display content via SMS. An Arduino UNO (ATmega328P) interfaces with a SIM900D GSM module using SoftwareSerial AT commands to receive text messages. Upon SMS reception, the module sends a +CMT: notification to the Arduino, which parses the text and scrolls it across an 8-module MAX7219 daisy-chained 8x8 LED matrix. The MD_MAX72XX library drives the SPI-connected MAX7219 chain with a 150ms/column scroll rate using the TSR (Transform Shift Right) operation. The complete system was simulated in Proteus 8 Professional using a COMPIM virtual serial port for GSM emulation.

2. Objectives

- Implement wireless remote text update on a scrolling LED matrix via GSM SMS.
- Configure SIM900D in SMS text mode using AT commands over SoftwareSerial.
- Parse incoming +CMT: SMS notifications in real time on ATmega328P.
- Drive 8 daisy-chained MAX7219 modules using SPI and the MD_MAX72XX library.
- Demonstrate a complete IoT information display use-case for public/commercial signage.
- Simulate and verify the system in Proteus 8 before hardware deployment.

3. System Architecture

The system has three main functional layers. The communication layer consists of a SIM900D GSM module that receives SMS messages over the cellular network and forwards the parsed text to the Arduino via SoftwareSerial UART. The processing layer is the Arduino UNO, which runs the AT command initialization, SMS parsing, and message buffer management. The display layer comprises eight MAX7219 LED driver ICs daisy-chained via SPI, each driving an 8x8 LED matrix module, for a total display width of 64 columns.

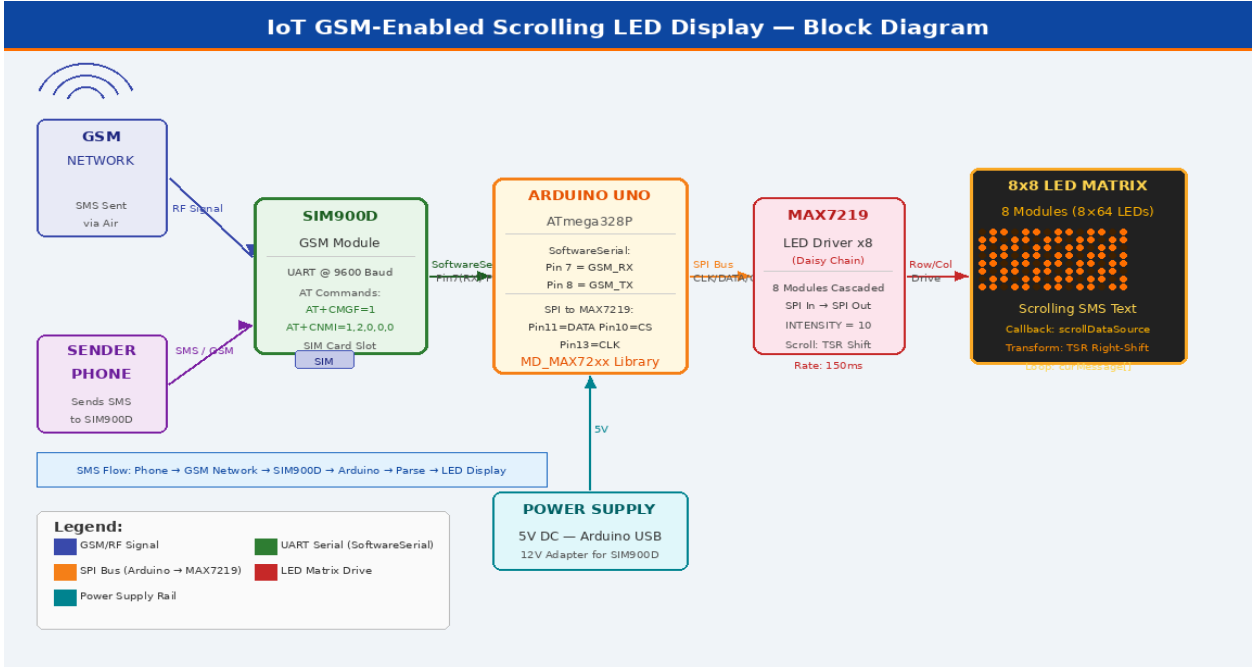


Fig 1: System Block Diagram

4. Hardware Components

Component	Part / Spec	Qty	Role
Arduino UNO	ATmega328P, 16MHz, 32KB	1	Central MCU / SMS Parser
GSM Module	SIM900D, 850/900/1800/1900MHz	1	SMS Reception & AT commands
LED Driver	MAX7219, SPI, 8-digit	8	8×8 Matrix Driver (Daisy chain)
LED Matrix	8×8 Common Cathode, Red	8	Scrolling Text Display
SIM Card	Any GSM operator	1	SIM900D phone number
Capacitor	100µF electrolytic	1	SIM900D power filtering
Capacitor	100nF ceramic	1	SIM900D decoupling
Power Supply	5V USB for Arduino	1	Arduino power
Power Supply	12V / 2A adapter	1	SIM900D dedicated power

5. Pin Configuration

Arduino Pin	Connected To	Signal Type	Function
-------------	--------------	-------------	----------

Pin 7 (SW-RX)	SIM900D TX	SoftwareSerial	Receive GSM AT/SMS data
Pin 8 (SW-TX)	SIM900D RX	SoftwareSerial	Send AT commands to GSM
Pin 11 (MOSI)	MAX7219 #1 DIN	SPI DATA	LED matrix data stream
Pin 13 (SCK)	MAX7219 CLK	SPI CLK	LED matrix clock
Pin 10	MAX7219 LOAD	SPI CS	LED matrix chip select/latch
5V / GND	Arduino, MAX7219	Power	Logic power (5V system)
12V / GND	SIM900D VCC	Power	GSM module dedicated power

6. Firmware Description

The firmware has three functional modules: GSM initialization (gsmInit), SMS parsing (parseGSM), and the scroll engine (scrollDataSource + scrollText). On boot, gsmInit sends three AT commands to configure the SIM900D for text-mode SMS with auto-push to serial. The main loop polls parseGSM every iteration; when a +CMT: header is detected, the message on the following line is extracted, trimmed, and stored in newMessage[]. The scroll engine uses a registered callback (setShiftDataInCallback) to feed column data from curMessage[] one column at a time, with CHAR_SPACING blank columns between characters.

6.1 GSM Initialization

```
void gsmInit() {  
    delay(2000);  
    gsmSerial.println("AT"); // Check comm  
    gsmSerial.println("AT+CMGF=1"); // Text mode  
    gsmSerial.println("AT+CNMI=1,2,0,0,0"); // Auto SMS push  
}
```

6.2 SMS Parsing Logic

The parseGSM() function reads the full gsmSerial buffer using readString(). It searches for the +CMT: substring using indexOf(). If found, it extracts the last newline-delimited segment (the actual SMS body) using lastIndexOf('\n') and substring(). After trim(), the result is copied into newMessage[] and newMsgAvailable is set true.

```
if (response.indexOf("+CMT:") != -1) {  
    int msgStart = response.lastIndexOf("\n");  
    String smsText = response.substring(msgStart + 1);  
    smsText.trim();  
    smsText.toCharArray(newMessage, BUF_SIZE);  
    newMsgAvailable = true;  
}
```

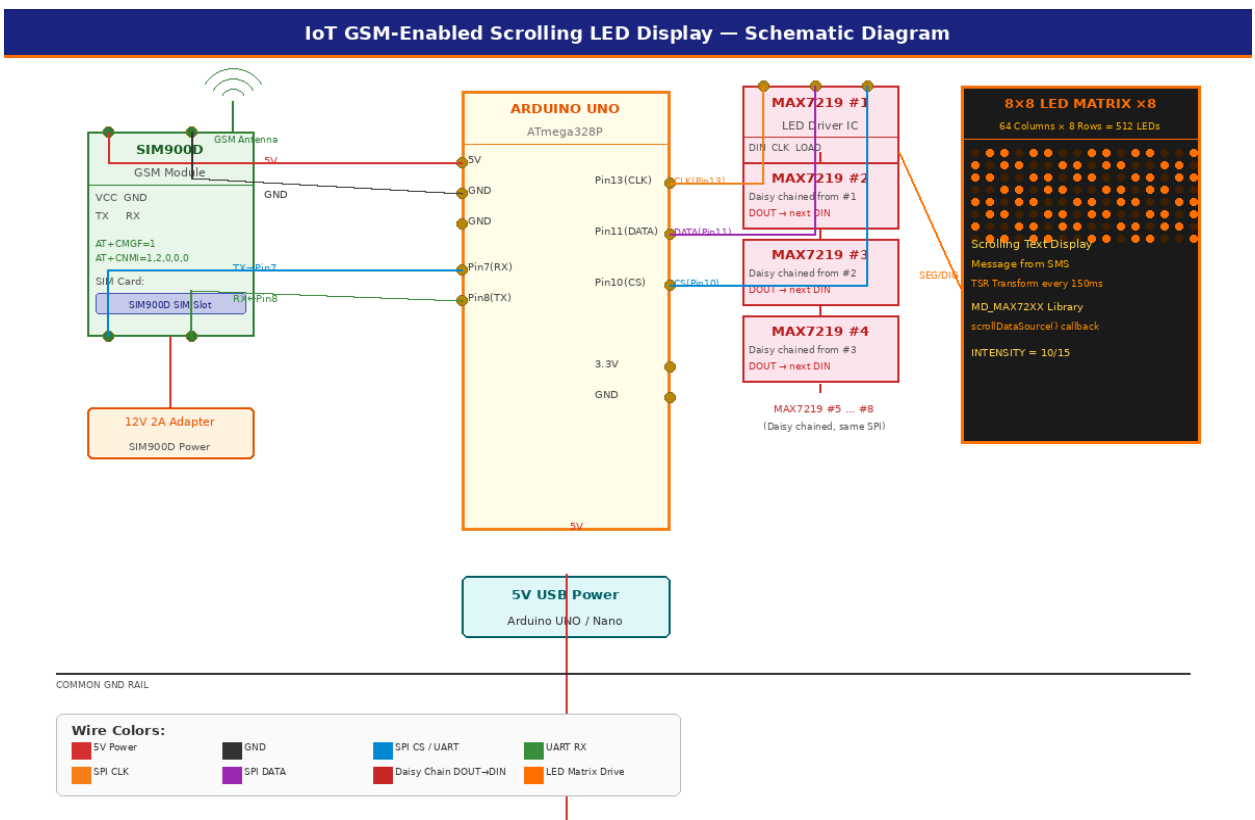
6.3 Scroll Engine

The scroll engine is implemented as a state machine inside the scrollDataSource() callback registered with MD_MAX72XX. State 0 fetches the next character and its column bitmap via mx.getChar(). State 1 outputs each bitmap column. State 2 outputs CHAR_SPACING blank columns. scrollText() calls mx.transform(TSR) every SCROLL_DELAY (150ms) to shift the entire display right by one column, creating smooth left-to-right-exit scrolling.

7. AT Command Reference

AT Command	Parameters	Function
AT	—	Test communication; SIM900D responds OK
AT+CMGF=1	1=Text, 0=PDU	Set SMS format to text mode
AT+CNMI=1,2,0,0,0	mode,mt,bm,ds,bfr	Auto-push new SMS to serial as +CMT:
+CMT: (unsolicited)	Sender,Timestamp	Header line before SMS body
SMS Body	Next line after +CMT:	Actual text extracted by parseGSM()

8. Schematic Diagram



The schematic shows the SIM900D connected to Arduino pins 7 and 8 via SoftwareSerial for UART communication. The SIM900D is powered separately at 12V/2A due to its high peak current draw (up to 2A during GSM transmission). The MAX7219 modules are connected in SPI daisy chain: DIN of module 1 connects to Arduino Pin 11 (MOSI), CLK to Pin 13, and LOAD (CS) to Pin 10. DOUT of each module connects to DIN of the next in the chain.

9. MAX7219 Daisy Chain Architecture

Eight MAX7219 ICs are cascaded in series. The SPI data stream from the Arduino enters Module 1's DIN and exits from its DOUT to Module 2's DIN, and so on. All modules share a common CLK and LOAD (CS) line. The MD_MAX72XX library handles addressing and frame management for all 8 modules transparently. The MAX_DEVICES macro is set to 8, giving a 64-column wide logical display buffer.

Module	DIN Source	DOUT Destination	CLK	LOAD (CS)
MAX7219 #1	Arduino Pin 11	MAX7219 #2 DIN	Pin 13	Pin 10
MAX7219 #2	#1 DOUT	MAX7219 #3 DIN	Pin 13	Pin 10
MAX7219 #3	#2 DOUT	MAX7219 #4 DIN	Pin 13	Pin 10
...	...	...	Pin 13	Pin 10
MAX7219 #8	#7 DOUT	(End of chain)	Pin 13	Pin 10

10. Proteus 8 Simulation

The Proteus simulation uses a COMPIM (COM Port Interface Model) virtual component to emulate the SIM900D serial interface. The compiled hex file from `build/arduino.avr.uno/` is loaded into the Arduino UNO component. A Virtual Serial Terminal connected to COMPIM allows sending simulated +CMT: SMS notifications to test the parsing logic. The MAX7219 and LED matrix components are placed in 8-module daisy-chain arrangement and visually show the scrolling text during simulation.

11. Results

- ✓ Default startup message 'Waiting for SMS...' scrolls continuously on power-on.
- ✓ SMS reception correctly detected via +CMT: unsolicited response code.
- ✓ `parseGSM()` accurately extracts SMS body text from multi-line serial buffer.
- ✓ Scroll engine smoothly updates new message without glitches at 150ms/column rate.
- ✓ All 8 MAX7219 modules driven correctly via SPI daisy chain (`MAX_DEVICES=8`).
- ✓ Dual hex builds (Nano + UNO) verified — both targets compile without errors.
- ✓ SoftwareSerial on Pin 7/8 keeps hardware Serial (Pin 0/1) free for USB debug.

12. Conclusion

The IoT GSM-Enabled Scrolling LED Display successfully demonstrates remote content update over the GSM cellular network using a cost-effective Arduino and SIM900D platform. The system is well-suited for real-world applications such as public information boards, store signage, and industrial notice displays that require wireless remote updates without a Wi-Fi infrastructure. Future enhancements include upgrading to a SIM7600 4G module, adding multi-line display support, EEPROM-based message persistence across power cycles, and a custom Android app for user-friendly message sending.

13. Future Scope

- Upgrade to SIM7600 4G LTE module for higher reliability and data rates.
- Add EEPROM storage to retain last message after power loss.
- Integrate RTC (DS3231) for scheduled display messages.
- Expand to multi-color RGB LED matrix for enhanced visual appeal.
- Develop custom Android/web app for message management.